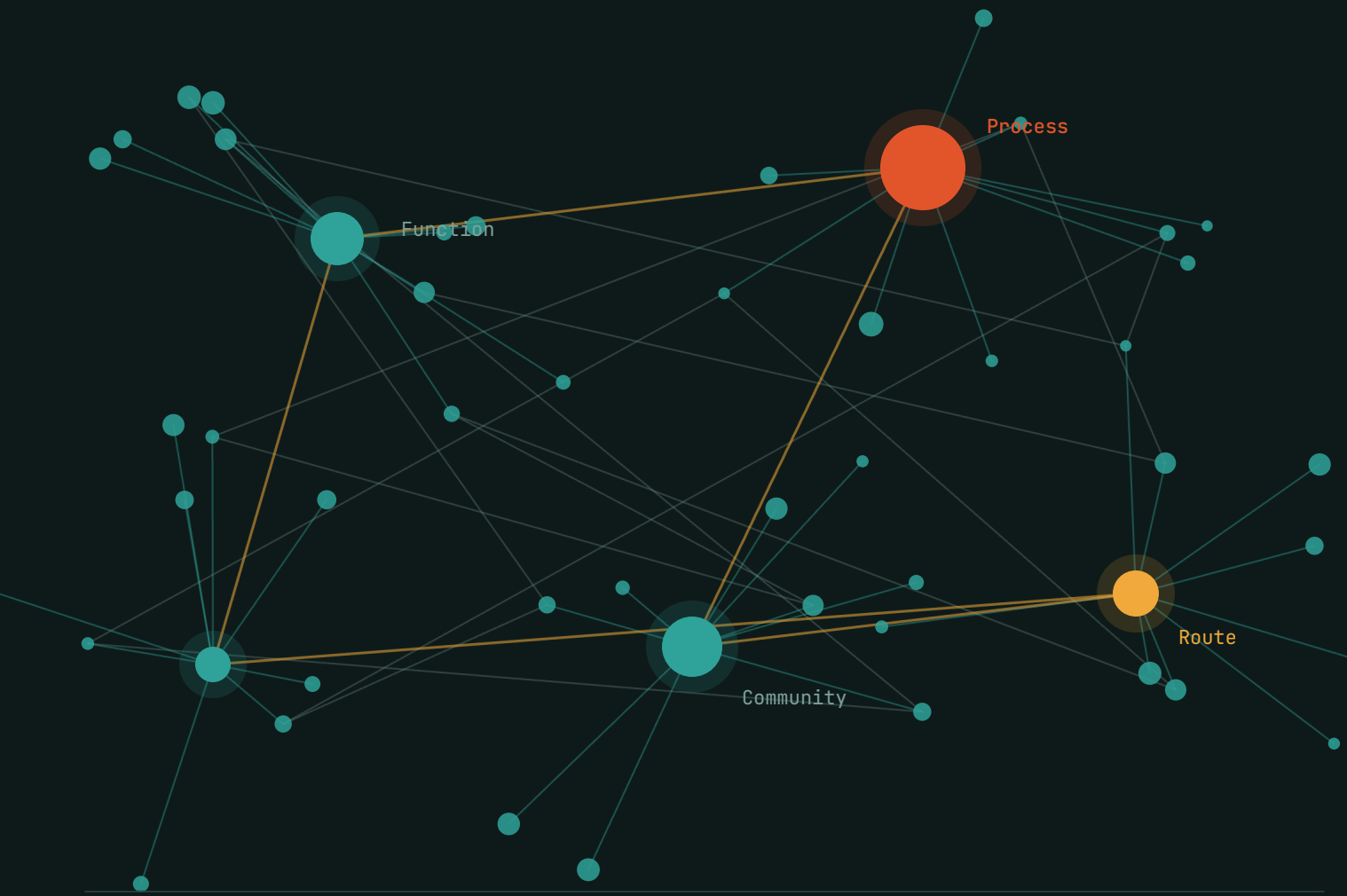
BÁO CÁO KỸ THUẬT

24 · 10 · 2026

GitNexus

Biến cả codebase thành đồ thị tri thức mà AI agent có thể truy vấn



PHIÊN BẢN NPM
1.6.12

COMMIT ĐÃ ĐỌC
d8a256a

NGÔN NGỮ · TOOL MCP
18 · 19

GIẤY PHÉP
PolyForm NC

Nguồn: github.com/abhigyanpatwari/GitNexus (bản shallow clone ngày 24/10/2026). Báo cáo tổng hợp từ [illegible] và tài liệu của repo; công cụ chưa được chạy thử.

moda.app

01 · TÓM TẮT ĐIỀU HÀNH

GitNexus là gì, và vì sao đáng chú ý

GitNexus lập chỉ mục cả một codebase thành đồ thị tri thức gồm symbol, lời gọi, phụ thuộc và luồng thực thi. Coding agent truy vấn đồ thị đó qua MCP, CLI hoặc giao diện web thay vì tự grep. Bộ lập chỉ mục chạy cục bộ và không gọi LLM.

248.612
symbol trong chỉ mục của chính repo (AGENTS.md)

565.510
quan hệ (lời gọi, import, kế thừa...) giữa các symbol

918
luồng thực thi (process) được tự phát hiện

18
ngôn ngữ có provider, kể cả COBOL và Vue

19
công cụ MCP (17 theo repo + 2 theo nhóm)

35 · 33
bảng node và loại quan hệ (đếm từ mã nguồn)

Là gì

Một monorepo TypeScript: gói npm gitnexus (CLI, MCP server, API HTTP, bộ lập chỉ mục), giao diện web React, và gói kiểu dùng chung. Phiên bản npm 1.6.12, giấy phép PolyForm Noncommercial 1.0.0, yêu cầu Node 22.18+ hoặc 24.11+.

Hoạt động thế nào

Lệnh analyze quét repo, parse bằng tree-sitter, phân giải lời gọi và kế thừa, gom cụm bằng thuật toán Leiden, dò luồng thực thi, rồi nạp toàn bộ vào cơ sở dữ liệu đồ thị LadybugDB nằm trong thư mục .gitnexus/. Agent hỏi đồ thị đã tính sẵn.

Dùng vào việc gì

Hỏi ai gọi hàm này, đo phạm vi ảnh hưởng trước khi sửa (impact), ánh xạ git diff sang symbol bị tác động (detect_changes), đổi tên theo ngữ nghĩa (rename), và truy vết dòng dữ liệu nguy hiểm (explain, cần cờ --pdg).

Nhận định tổng quát

ĐIỂM MẠNH

- Kỷ luật "chính xác hơn đầy đủ": bỏ sót một route chỉ là giới hạn độ phủ, còn bịa ra một route là nói sai sự thật (ARCHITECTURE.md).
- Kết quả luôn kèm độ tin cậy: epistemic exact hoặc lower-bound, và risk UNKNOWN thay cho con số 0 giả.
- Một đồ thị, ba giao diện (MCP, CLI, web), cùng 12 skill dựng sẵn cho agent.

CẦN LƯU Ý

- Giấy phép phi thương mại: dùng cho mục đích thương mại cần giấy phép riêng từ akonlabs.com.
- gitnexus serve chưa tự xác thực; triển khai công khai phải đặt sau proxy có token.
- ARCHITECTURE.md lệch mã nguồn: ghi 19 phase, 16 tool, 44 loại node; mã thực tế có 20 phase, 19 tool, 35 loại node.

moda.app

02 · Ý TƯỞNG CỐT LÕI

Tính sẵn đồ thị, agent khỏi phải đoán

AI coding agent thường sửa một hàm mà không biết còn bao nhiêu nơi phụ thuộc vào nó. GitNexus xử lý bằng cách tính trước mọi phụ thuộc, chuỗi lời gọi và luồng thực thi vào một đồ thị có thể truy vấn (theo README của gói gitnexus).

Agent chỉ có grep và đọc file

- Tìm theo chuỗi ký tự rồi tự suy ra ai gọi ai.
- Dễ sửa một hàm mà bỏ sót nơi gọi gián tiếp.
- Phải khám phá cấu trúc bằng nhiều lần đọc file.

Agent có đồ thị GitNexus

- Hỏi thẳng bằng query, context, impact, trace.
- Biết nơi gọi, luồng thực thi và cụm liên quan.
- Nhận kèm độ tin cậy, cảnh báo khi dữ liệu thiếu.

Ba cách dùng cùng một đồ thị

MCP

Qua công cụ của agent

Agent gọi tool như query, context, impact. Nhiều agent dùng chung một server HTTP trên cổng 3111.

CLI

Qua dòng lệnh

Lệnh query, impact, status, list. Cờ analyze --watch lập chỉ mục lại khi file thay đổi.

WEB UI

Qua trình duyệt

Chạy gitnexus serve rồi mở giao diện web để duyệt đồ thị và hỏi đáp AI trên index sẵn có.

Bắt đầu nhanh

```
# 1. Lập chỉ mục từ thư mục gốc của repo
npx gitnexus analyze
# 2. Cấu hình MCP cho các editor (chạy một lần)
npx gitnexus setup
# 3. Hỏi đồ thị bằng CLI, hoặc phục vụ index cho web UI
gitnexus impact UserService --direction upstream
gitnexus serve
```

Sáu việc agent làm được ngay

Việc cần làm	Công cụ
Tìm code và các luồng thực thi liên quan	query
Xem người gọi và phụ thuộc của một symbol	context
Kiểm tra một thay đổi có thể ảnh hưởng gì	impact
Dò đường đi giữa hai symbol	trace
Đánh giá tác động của một git diff	detect_changes
Xem trước và áp dụng đổi tên theo ngữ nghĩa	rename

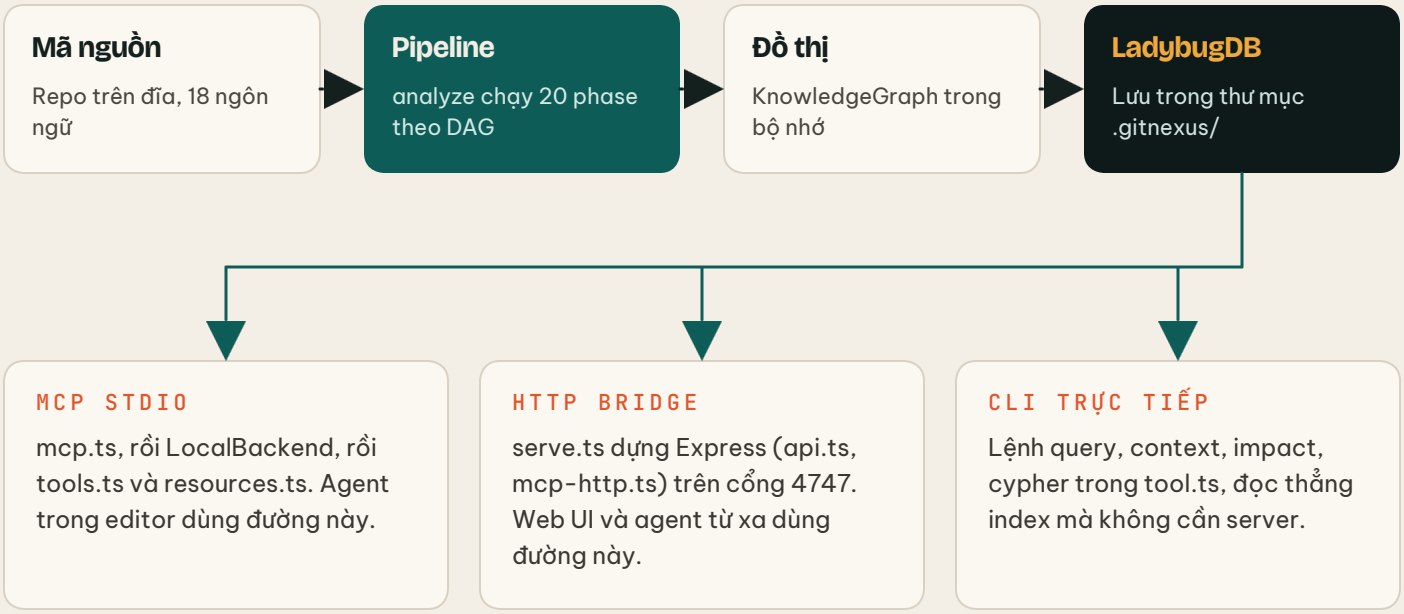
moda.app

03 · KIẾN TRÚC TỔNG THỂ

Một monorepo, một đồ thị, ba cửa vào

Repo gồm gói CLI và MCP (gitnexus), web UI mỏng (gitnexus-web), gói kiểu dùng chung (gitnexus-shared), cùng các gói tích hợp agent và bộ đánh giá. Mọi thứ xoay quanh một luồng duy nhất: lập chỉ mục, đồ thị, công cụ truy vấn.

Luồng từ mã nguồn đến công cụ



Bản đồ thư mục

Đường dẫn	Vai trò
gitnexus/	Gói npm gitnexus: CLI, MCP server, API HTTP, pipeline, LadybugDB
gitnexus-web/	Web UI Vite và React, client mỏng gọi API của gitnexus serve
gitnexus-shared/	Kiểu và hằng số dùng chung cho CLI và web (schema, ngôn ngữ)
gitnexus-claude-plugin/ +2 gói	Skill, hook và cấu hình tĩnh cho Claude, Cursor, Factory
eval/	Bộ đánh giá SWE-bench: agent có và không có GitNexus
.github/	32 workflow: CI, quét bảo mật, phát hành, review tự động

Muốn sửa gì, bắt đầu từ đâu (đường dẫn trong gitnexus/)

Dựng đồ thị: `src/core/ingestion/pipeline-phases/`
Schema và DB: `src/core/lbug/`
Tìm kiếm: `src/core/search/`
Công cụ MCP: `src/mcp/tools.ts`
Nhóm đa repo: `src/core/group/`
Embeddings: `src/core/embeddings/`

moda.app

04 · PIPELINE LẬP CHỈ MỤC

20 phase xếp thành một DAG

Lệnh analyze gọi runFullAnalysis, rồi runPipelineFromRepo. Mỗi phase khai báo phụ thuộc và kiểu đầu ra; bộ chạy sắp xếp theo thứ tự topo, từ chối chu trình và chỉ đưa cho phase kết quả của đúng các phụ thuộc đã khai.

Phụ thuộc giữa các phase

● nối tiếp ○ độc lập ○ chỉ khi --pdg

01	Quét cấu trúc	scan → structure
02	Nhánh độc lập 1	springConfig, markdown, cobol
03	Cú pháp	parse
04	Nhánh độc lập 2	routes, tools, orm
05	Liên kết file	crossFile → scopeResolution
06	Lớp Spring	springAutoConfiguration, springAop, springDestinations
07	Dọn cục bộ	pruneLocalSymbols
08	Kế thừa và DI	mro → springAopInheritance → di
09	Bậc cao	communities → processes
+2	Cờ --pdg	taintSummaries, callSummaries

Vòng đời của một lần analyze



Trước khi chạy, analyze thoát sớm nếu lastCommit trùng HEAD (trừ khi dùng --force) và lưu lại embeddings cũ. Sau 85% là đánh chỉ mục FTS và khôi phục embeddings; embeddings mới (cờ --embeddings) chiếm 90–98%; 98–100% ghi metadata, đăng ký repo rồi sinh AGENTS.md và CLAUDE.md.

Bộ chạy phase (runner)

- Sắp xếp topo (Kahn); từ chối tên trùng, phụ thuộc thiếu và chu trình.
- Chạy tuần tự; mỗi phase chỉ thấy kết quả của phụ thuộc đã khai báo.
- Mọi phase cùng ghi vào một KnowledgeGraph dùng chung.
- Thêm phase: tạo file, export, đăng ký trong buildPhaseList().

Parse theo mẻ, có cache

- Chia file thành các mẻ khoảng 20 MB để chặn bộ nhớ.
- Worker pool là đường parse duy nhất, không còn chế độ tuần tự.
- ParsedFile được stream qua kho trên đĩa tới bước phân giải.
- Cache nhận diện theo nội dung (SHA-256): mẻ file không đổi thì bỏ qua parse.

moda.app

05 · PHÂN GIẢI VÀ ĐA NGÔN NGỮ

Một pipeline, mười tám ngôn ngữ

Phần dày nhất của tài liệu kiến trúc dành cho một câu hỏi: lời gọi này trỏ tới hàm nào? GitNexus tách phần chung (phân giải theo phạm vi) khỏi phần riêng của từng ngôn ngữ, rồi trả đồ thị kèm độ tin cậy.

Bốn tầng nuôi một đồ thị



18 provider đã đăng ký

JavaScript	TypeScript	Python
Java	C	C++
Objective-C	C#	Go
Ruby	Rust	PHP
Kotlin	Swift	Dart
Vue	COBOL	Zig

COBOL (viền cam) dùng bộ xử lý regex riêng, không qua tree-sitter.

Thứ tự phát cạnh khi phân giải

1 emitReceiverBoundCalls (đầu tiên)
2 emitFreeCallFallback
3 emitReferencesViaLookup
4 emitPropertyDispatchCalls
5 emitCallableValueFlow
6 emitImportedValueReferences
7 emitUniquePropertyAccesses (phương án cuối)
8 emitImportEdges

Thứ tự kế thừa (MRO)

first-wins: Java, C#, C++, TypeScript, Ruby, Go

c3: Python (tuyến tính hoá C3)

ruby-mixin: Ruby, nhận biết mixin

none: ngôn ngữ đơn kế thừa

Mỗi ngôn ngữ cắm vào bằng ScopeResolver và đăng ký trong SCOPE_RESOLVERS; mã dùng chung trong ingestion/ không được nêu tên ngôn ngữ (quy tắc trong AGENTS.md).

Nguyên tắc: chính xác hơn đầy đủ

- Bỏ sót một route là giới hạn độ phủ; bịa ra một route là nói dối. Regex không dịch chính xác thì bị bỏ.
- Tên mơ hồ thì từ chối chọn, không đoán: tên khai báo hai lần với giá trị khác nhau bị loại.
- Một đối tượng tên app hay router không phải bằng chứng: Express chỉ được nhận diện qua import và nguồn gốc khai báo.
- Vượt trần 32 mục tiêu thì không phát cạnh một phần, chỉ cảnh báo kèm số liệu bị bỏ.
- impact và context trả epistemic exact hoặc lower-bound, để biết con số là chính xác hay chỉ là cận dưới.
- Chỉ lời gọi chưa gán được kiểu trong chương trình mới làm số đếm hụt; lời gọi tới thư viện ngoài (như fetch) được báo nhưng không làm hụt.

moda.app

Nguồn: ARCHITECTURE.md (Scope-Resolution Pipeline); gitnexus-shared/src/languages.ts

06 · DỮ LIỆU VÀ LƯU TRỮ

35 loại node, một bảng quan hệ

Đồ thị nằm trong LadybugDB, cơ sở dữ liệu đồ thị nhúng có hỗ trợ vector (tên cũ KuzuDB). Mỗi loại node có bảng riêng; mọi quan hệ dùng chung bảng CodeRelation và phân biệt bằng cột type.

Các nhóm node

Cấu trúc	File, Folder, Module, Namespace, Section
Khả gọi	Function, Method, Constructor, Delegate, Macro, Template
Kiểu	Class, Interface, Struct, Enum, Trait, Impl, Union, Typedef, TypeAlias, Record, Protocol, Category, Annotation
Giá trị	Const, Static, Variable, Property, CodeElement
Lớp phân tích	Community, Process
Lớp framework	Route, Tool, Destination
Cờ --pdg	BasicBlock

33 loại quan hệ trong CodeRelation

Cấu trúc	CONTAINS, DEFINES, HAS_METHOD, HAS_PROPERTY
Gọi, phụ thuộc	CALLS, IMPORTS, ACCESSES, FETCHES, QUERIES, WRAPS, INJECTS
Kế thừa	EXTENDS, IMPLEMENTS, METHOD_OVERRIDES, OVERRIDES, METHOD_IMPLEMENTS
Lớp phân tích	MEMBER_OF, STEP_IN_PROCESS, ENTRY_POINT_OF
Framework	HANDLES_ROUTE, HANDLES_TOOL, PUBLISHES_TO, CONSUMES_FROM, CONDITIONAL_ON, DECLARES, ADVISED_BY
Cờ --pdg	CFG, REACHING_DEF, CDG, POST_DOMINATE, TAINTED, SANITIZES, TAINT_PATH

Lưu trữ và độ tươi của chỉ mục

```
<repo>/.gitnexus/
  lbug LadybugDB
  lbug.wal, lbug.shadow nhật ký, checkpoint
  lbug.lock khoá một người ghi
  gitnexus.json lastCommit, stats
  meta.json bản sao cũ
~/.gitnexus/
  registry.json đăng ký repo cho MCP
  stores/<key>/ kho dùng chung
    caches/ parse cache
    commits/ 1 đồ thị mỗi commit
    checkouts/ đồ thị cục bộ
```

- **Tăng dần theo mặc định:** chỉ ghi lại hàng của file đổi (cùng importer của chúng); quá khoảng 50% file thì ghi lại toàn bộ.
- **Dùng chung kho:** worktree và các clone cùng origin chia sẻ một kho; tắt bằng --no-share hoặc GITNEXUS_SHARED_STORE=off.
- **Độ tươi:** so lastCommit với HEAD thành current, behind, diverged hoặc unknown, cùng một payload cho MCP và web.
- **Embeddings:** Snowflake arctic-embed-xs 384 chiều, chỉ cho File, Function, Class, Method, Interface; cập nhật theo SHA1 nội dung.

moda.app

07 · TÌM KIẾM VÀ PHÂN TÍCH

Từ đồ thị thô đến câu trả lời xếp hạng

Ngoài các cạnh, bước phân tích bậc cao tạo thêm hai lớp giúp agent định hướng: cụm chức năng (Community) và luồng thực thi (Process). Phía truy vấn trộn từ khoá với ngữ nghĩa, còn impact chấm rủi ro bằng ngưỡng cố định.

Tìm kiếm lai: BM25 và vector

```
điểm = Σ 1 / (60 + hạng)
```

Từ khoá (BM25) và ngữ nghĩa (vector) mỗi bên cho một danh sách xếp hạng; điểm cộng dồn theo hạng, không cần chuẩn hoá. Kết quả có mặt ở cả hai bên được cộng điểm.

Cụm chức năng: thuật toán Leiden

Chạy Leiden (bản graphology được vendor sẵn trong repo) trên các cạnh CALLS để gom code hay gọi nhau thành cụm. Agent điều hướng theo khu chức năng thay vì theo thư mục. Engine native icebug có sẵn nhưng còn thử nghiệm và cho ID cụm khác.

Luồng thực thi (Process) được dò thế nào

1 · Tìm entry point → **2 · Duyệt theo CALLS** → **3 · Gộp và gắn nhãn**

Hàm không có người gọi nội bộ, chấm điểm theo tên (handle, on, Controller), trạng thái export và framework; hàm tiện ích bị trừ điểm.

DFS theo cạnh CALLS, sâu tối đa 10 bước, rẽ tối đa 4 nhánh mỗi node; chỉ giữ luồng từ 3 bước trở lên.

Gộp luồng trùng lặp, gắn nhãn heuristic (ví dụ HandleLogin → CreateSession) và ghi nhận luồng nằm trong một cụm hay đi xuyên cụm.

Chỉnh được bằng --max-processes, --max-process-trace-depth, --max-process-branching, --max-entry-point-candidates. Khi không đặt, số process tối đa theo công thức động symbols/10.

Thang rủi ro của impact (ngưỡng trong gitnexus-shared)

LOW
Dưới mọi ngưỡng còn lại.

MEDIUM
5 gọi trực tiếp trở lên, hoặc 30 symbol bị ảnh hưởng trở lên.

HIGH
15 trực tiếp, 3 process, 3 module hoặc 100 bị ảnh hưởng trở lên.

CRITICAL
30 trực tiếp, 5 process, 5 module hoặc 200 bị ảnh hưởng trở lên.

UNKNOWN
Upstream không thấy người gọi, hoặc mẫu chưa đầy đủ.

UNKNOWN không có nghĩa là an toàn. Danh sách người gọi rỗng có thể chỉ cho thấy chỉ mục không phân giải được lời gọi (truy cập thuộc tính trên object thường, dispatch động, gọi chéo ngôn ngữ). Khi mẫu kết quả chưa đầy đủ, mức LOW hoặc MEDIUM cũng bị hạ xuống UNKNOWN thay vì báo một con số tự tin.

Chính repo ép agent theo quy tắc này (AGENTS.md)

- Chạy impact trước khi sửa một symbol; HIGH hoặc CRITICAL phải được cảnh báo.
- Coi risk UNKNOWN là chưa giải quyết: danh sách người gọi rỗng không chứng minh symbol không được dùng.
- Chạy detect_changes trước khi commit; kết quả partial hoặc truncated không phải kết quả sạch.

moda.app

08 · CÔNG CỤ MCP

19 công cụ, chia theo việc cần làm

Server MCP phơi 19 công cụ, 10 resource và 2 prompt dẫn đường. Công cụ tra cứu nhận tham số repo tuỳ chọn, và query, context, impact còn đi theo nhóm repo bằng cú pháp @group.

KHÁM PHÁ · 5

`list_repos`: liệt kê repo đã lập chỉ mục

`query`: tìm lai BM25 và vector, gom theo process

`context`: góc nhìn 360 độ về người gọi, phụ thuộc, process

`trace`: đường đi có hướng ngắn nhất giữa hai symbol

`cypher`: truy vấn Cypher thô lên đồ thị

Công cụ theo repo nhận thêm tham số branch cho chỉ mục ghim bằng analyze --branch (trừ read_file và grep, vốn đọc thẳng checkout).

AN TOÀN KHI SỬA · 4

`impact`: phạm vi ảnh hưởng theo độ sâu, kèm mức risk

`detect_changes`: ánh xạ git diff sang symbol và process bị ảnh hưởng

`rename`: đổi tên theo ngữ nghĩa, xem trước rồi áp dụng bằng plan_token dùng một lần

`check`: kiểm tra cấu trúc, như chu trình import ép thứ tự khởi tạo

Công cụ ghi như rename cần chỉ rõ repo khi có nhiều repo mà chưa đặt repo mặc định.

API VÀ TÍCH HỢP · 4

`route_map`: route, handler và nơi gọi

`api_impact`: báo cáo tác động trước khi đổi một handler

`shape_check`: đối chiếu shape phản hồi với cách client đọc thuộc tính

`tool_map`: định nghĩa tool MCP/RPC và handler

LUỒNG DỮ LIỆU, CẦN --PDG · 2

`explain`: luồng taint từ nguồn đến đích

`pdg_query`: phụ thuộc điều khiển và dữ liệu

ĐỌC MÃ GỐC · 2

`read_file`: đọc file trong checkout, giới hạn dòng

`grep`: dự phòng khi đồ thị chưa đủ

NHÓM REPO · 2

`group_list`: liệt kê nhóm đã cấu hình. `group_sync`: dựng lại Contract Registry và liên kết repo.

Kiểm soát truy cập và ngân sách phản hồi

- `GITNEXUS_MCP_READ_ONLY=1` chỉ giữ bề mặt đọc một repo; rename chỉ xem trước, cypher và nhóm repo bị ẩn.
- `GITNEXUS_MCP_ALLOWED_REPOS` giới hạn repo cho tool, resource và HTTP nhúng.
- `maxTokens` cắt phản hồi của query, context, impact theo ước lượng 4 byte mỗi token.
- `response_sections` chọn phần trả về; phản hồi vẫn ghi rõ phần bị lược bỏ.

```
Resource: repos, setup, repo/{name}/context, clusters, cluster/{name}, processes, process/{name},
schema, group/{name}/contracts, group/{name}/status
Prompt: detect_impact, generate_map
```

moda.app

Nguồn: docs/mcp-reference.md; gitnexus/src/mcp/tools.ts

09 · PHÂN TÍCH SÂU

Luồng dữ liệu và nhiều repo

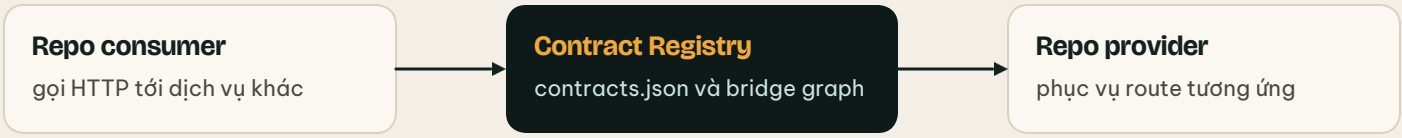
Hai năng lực vượt ra ngoài đồ thị lời gọi: lớp CFG và PDG cấp câu lệnh để lần vết taint, và nhóm repo để lần theo hợp đồng giữa các dịch vụ. Cả hai đều là tuỳ chọn.

Các lớp CFG và PDG (tuỳ chọn)

M1 CFG	BasicBlock và cạnh CFG; loại cạnh (seq, cond-true, loop-back...) nằm ở cột reason của bảng CodeRelation.
M2 REACHING_DEF	Phụ thuộc dữ liệu từ nơi gán đến nơi dùng, tính bằng bộ giải điểm bất động; tên biến nằm ở cột reason.
M3-4 TAINT	TAINTED, SANITIZES, TAINT_PATH: taint trong và giữa các hàm, từ nguồn đến đích. Đây là dữ liệu của tool explain.
M5 CDG	Phụ thuộc điều khiển kiểu Ferrante trên cây hậu trội; nhãn nhánh T hoặc F nằm ở cột reason.
M6 ĐỌC	pdg_query trả lời điều kiện để chạy (controls) và dòng chảy của biến (flows); explain đọc taint. Luôn neo theo điểm và có LIMIT.

Mặc định tắt, được tính vào khoá cache và bị chặn bởi trần số vùng và cạnh. Provider báo vùng không hỗ trợ thay vì bịa ra luồng đầy đủ. Các cạnh PDG cố ý nằm ngoài VALID_RELATION_TYPES mặc định và schema web.

Nhóm repo và hợp đồng giữa dịch vụ



ContractLink: HTTP consumer đến provider, nối theo Contract.symbolUid

- **Định tuyến:** query, context, impact nhận repo là @tên-nhóm (hoặc @nhóm/thư-mục-member). query gộp kết quả từng repo bằng RRF.
- **Lan tác động:** impact chạy trong member được chọn rồi lan qua biên bằng Contract Bridge. trace nối đường đi qua một biên hợp đồng (hop CONTRACT_LINK), bị chặn ở một biên.
- **Luồng dữ liệu (thử nghiệm):** với pdg: true, mỗi đoạn kề biên được gắn luồng REACHING_DEF trong thủ tục; dữ liệu không đi xuyên biên repo.
- **Trạng thái:** lộ ra qua resource gitnexus://group/{name}/contracts và /status. Các tool group_query, group_context, group_impact cố ý không được thêm.

moda.app

10 · WEB UI, TRIỂN KHAI, BẢO MẬT

Web UI, triển khai và bảo mật

Web UI là client mỏng: mọi truy vấn đi qua HTTP API của gitnexus serve (cổng 4747). Backend chưa tự xác thực, nên ranh giới bảo mật nằm ở proxy đặt phía trước nó.

Ngăn xếp của web UI

Giao diện	React 19, Vite 8, Tailwind 4, i18next (đa ngôn ngữ)
Đồ thị	Sigma.js 3 vẽ bằng WebGL, graphology (layout ForceAtlas2), d3
Hỏi đáp AI	LangChain với Anthropic, OpenAI, Google GenAI và Ollama
Hiển thị	Mermaid, react-markdown, syntax highlighter
Kiểm thử	Vitest cho đơn vị, Playwright cho e2e (23.866 dòng TS và TSX trong src)

Ba cách triển khai

CỤC BỘ

gitnexus serve

Chạy backend trên máy, rồi mở gitnexus.vercel.app. Giao diện dùng index sẵn có, không cần tải lại hay lập chỉ mục lại.

DOCKER

docker compose up -d

Server ở cổng 4747, web ở cổng 4173. Hai image ký Cosign, phát hành đồng thời ở GHCR và Docker Hub.

RENDER

Blueprint hai dịch vụ

Server riêng tư, web công khai đóng vai proxy cho /api/*. Chi phí mặc định khoảng 35 USD mỗi tháng.

Ranh giới bảo mật cần nhớ

- **serve không tự xác thực.** Nó từ chối khởi động khi đặt GITNEXUS_PUBLIC_ORIGIN; muốn mở công khai phải đi qua proxy có token.
- **Trên Render, token là kiểm soát duy nhất.** Proxy xoá header Origin nên lớp chống CSRF của server không còn tác dụng; ai giữ token đọc được mọi repo đã index.
- **/api/mcp** dùng Bearer token so sánh hằng thời gian khi đặt GITNEXUS_MCP_AUTH_TOKEN; phải đặt cùng giá trị ở cả proxy và backend.
- **/api/grep chạy regex thật:** giới hạn 200 ký tự, tối đa 200 kết quả, ngân sách 5 giây trong worker có thể bị terminate.
- **Chuỗi cung ứng:** image chỉ phát hành từ tag vX.Y.Z khớp phiên bản npm, ký Cosign keyless kèm provenance và SBOM.
- **Quyền riêng tư dữ liệu:** bộ lập chỉ mục không gọi LLM; embeddings từ xa, wiki và AI chat chỉ dùng nhà cung cấp mà bạn cấu hình.

moda.app

11 · TÍCH HỢP AGENT

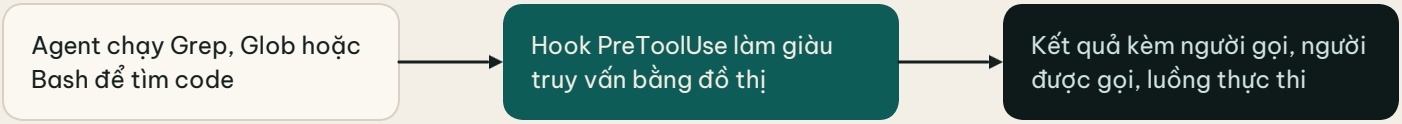
Cắm vào editor, dạy agent làm việc

analyze không chỉ lập chỉ mục: nó còn cài skill, đăng ký hook và ghi AGENTS.md hoặc CLAUDE.md vào repo, để agent dùng đồ thị mà không cần ai nhắc. Lệnh setup cấu hình MCP cho từng editor.

Mức hỗ trợ theo editor

Editor	MCP	Skill	Hook	Mức hỗ trợ
Claude Code	Có	Có	PreToolUse, PostToolUse	Đầy đủ
Cursor	Có	Có	postToolUse (cài tay)	Đầy đủ
Antigravity (Google)	Có	Có	AfterTool	Đầy đủ
Codex	Có	Có	PreToolUse, PostToolUse	Đầy đủ
Factory (Droid)	Có	Có	PostToolUse (plugin)	Đầy đủ
OpenCode, CodeBuddy, Qoder	Có	Có	Không	MCP và skill
Windsurf	Có, thủ công	Không	Không	Chỉ MCP

Hook của Claude Code làm gì



Hook PostToolUse chạy sau lệnh Bash để phát hiện chỉ mục đã cũ sau khi commit và nhắc agent lập chỉ mục lại. Cursor, Antigravity và Factory chỉ làm giàu kết quả sau khi chạy, không có gợi ý chỉ mục cũ.

12 skill đi kèm gói npm

CHUẨN · 8

```
gitnexus-guide
gitnexus-cli
gitnexus-exploring
gitnexus-debugging
gitnexus-impact-analysis
gitnexus-refactoring
gitnexus-pdg-query
gitnexus-taint-analysis
```

QUY TRÌNH · 4

```
gitnexus-plan
gitnexus-work
gitnexus-review
gitnexus-lfg (plan, work, review)
```

Tự kiểm chứng trong repo

- **eval/** chạy SWE-bench ở ba chế độ: baseline (grep, find, cat), native (thêm tool GitNexus) và native_augment (thêm làm giàu kết quả grep, được khuyến nghị).
- **pr-swarm-review/** gồm 7 persona chỉ đọc để rà soát PR, chạy song song (swarm) hoặc lần lượt (solo) từ bất kỳ AI CLI nào.
- **eval/workflow_bench** đo mức tiết kiệm token của bộ plan, work, review. Báo cáo này không chạy bộ đo nên không nêu con số nào.

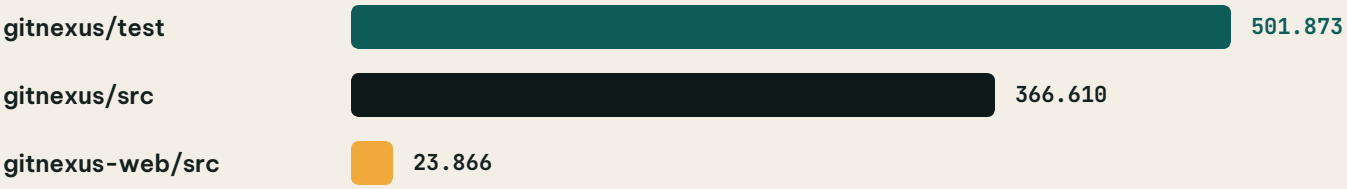
moda.app

12 · QUY MÔ, CHẤT LƯỢNG, GIẤY PHÉP

Mã kiểm thử nhiều hơn mã nguồn

Các số liệu dưới đây do báo cáo này tự đếm trên bản clone ở commit d8a256a (mọi file .ts và .tsx), trừ những mục ghi rõ nguồn tài liệu.

Khối lượng mã TypeScript (số dòng)



Mã kiểm thử bằng khoảng 1,37 lần mã nguồn của gói chính (501.873 so với 366.610 dòng).

905 + 434
file test unit và integration

32
workflow GitHub Actions

1.071
file mã nguồn trong gitnexus/src

12
skill đi kèm gói npm

7
persona review PR chỉ đọc

~200
test của web UI (theo AGENTS.md)

CI, bảo mật và phát hành

Chất lượng: ci-tests, ci-quality, ci-e2e, ci-report. Pre-commit chỉ chạy prettier và typecheck; test chạy ở CI.

Bảo mật: codeql, gitleaks, trivy, scorecard, dependency-review, workflow-lint.

Phát hành: publish, docker (chỉ từ tag khớp phiên bản npm), release-evaluation.

Tự động hoá: pr-autofix, pr-labeler, triage-sweep, skill-sync, gitnexus-review-agent, claude.

Giấy phép và mô hình thương mại

PolyForm Noncommercial 1.0.0

Dành cho mục đích phi thương mại (xem file LICENSE). Dùng thương mại cần giấy phép riêng, liên hệ qua akonlabs.com. Đây không phải tư vấn pháp lý.

Bản Enterprise (SaaS hoặc tự lưu trữ)

Review PR theo blast radius, wiki tự cập nhật, tự lập chỉ mục lại, đồ thị đa repo, hỗ trợ OCaml, ưu tiên ngôn ngữ và tính năng. Code Wiki cũng có ở bản OSS.

Khi đọc tài liệu của repo: CHANGELOG.md dừng ở bản 1.5.3 (1/4/2026) trong khi gói là 1.6.12, nên đừng dùng nó để suy ra lịch sử phát hành.

ARCHITECTURE.md ghi 19 phase, 16 tool, 44 loại node; mã thực tế có 20, 19 và 35.

moda.app

13 · ĐÁNH GIÁ VÀ PHỤ LỤC

Đánh giá và khuyến nghị

Tổng hợp từ việc đọc mã nguồn và tài liệu, chưa chạy thử công cụ. Phần dưới tách điều đã đối chiếu với mã khỏi điều chỉ đọc từ tài liệu.

Điểm mạnh

- Trung thực về độ chắc chắn: lower-bound, UNKNOWN, cảnh báo cắt cụt, không bịa route.
- Một đồ thị phục vụ MCP, CLI và web; schema và kiểu dùng chung giữa CLI và web.
- Hạ tầng kiểm thử và CI dày; repo tự áp quy tắc cho agent (AGENTS.md, hook, skill).
- Phủ 18 ngôn ngữ, có tài liệu khoảng trống riêng từng ngôn ngữ (docs/languages/).

Rủi ro và hạn chế

- Giấy phép PolyForm Noncommercial: cần kiểm tra trước khi dùng trong doanh nghiệp.
- serve chưa có xác thực; mở công khai chỉ an toàn khi có proxy giữ token.
- Độ phủ khác nhau theo ngôn ngữ; kết quả impact có thể chỉ là cận dưới, nên đọc cờ epistemic.
- Cài đặt cần Node 22.18+; không có prebuild thì cần trình biên dịch C/C++.
- Tài liệu kiến trúc lệch mã ở vài con số.

Nên thử thế nào

- Bắt đầu trên một repo nhỏ: analyze, rồi status, query và impact.
- Repo lớn: lập chỉ mục tốn RAM; bản Render mặc định có 2 GB, nâng lên 4 GB nếu hết bộ nhớ.
- Dùng agent theo quy tắc của AGENTS.md: impact trước khi sửa, detect_changes trước khi commit.
- Cần nhóm repo hoặc review PR tự động: xem bản Enterprise.

Phạm vi và giới hạn của báo cáo này

- **Nguồn:** bản clone nông (độ sâu 1) của abhigyanpatwari/GitNexus ở commit d8a256a, ngày 24/10/2026, gói gitnexus 1.6.12.
- **Đã làm:** đọc README, ARCHITECTURE, AGENTS, SECURITY, GUARDRAILS, docs/, package.json và các file lõi (schema, chấm rủi ro, tìm kiếm, cụm, luồng); tự đếm số dòng, số file, số loại node và quan hệ.
- **Chưa làm:** chạy analyze hay test, đo hiệu năng và độ chính xác phân giải. Số sao, số người đóng góp không lấy được vì API GitHub trả lỗi 403 qua proxy.
- **Số liệu tự báo của repo:** 248.612 symbol, 565.510 quan hệ, 918 luồng lấy từ khối GitNexus trong AGENTS.md; có thể đã cũ so với mã hiện tại.

FILE LÕI ĐÃ ĐỐI CHIẾU

`gitnexus-shared/src/lbug/schema-constants.ts · gitnexus-shared/src/impact-risk.ts · gitnexus/src/core/ingestion/pipeline.ts · core/search/hybrid-search.ts · core/ingestion/community-processor.ts · core/ingestion/process-processor.ts · mcp/tools.ts`

moda.app